DT081A Signal and Image Processing - Lab 3

Version: 1.1

Spatial domain image processing

Introduction

This laboratory session explores fundamental concepts in spatial domain image processing, building on the theoretical foundation from lectures 8-11. Through hands-on implementation and analysis, you will investigate how digital images are processed, from human perception aspects to advanced filtering techniques.

Install the required libraries:

```
pip install numpy opencv-python matplotlib
```

Example boiler plate code to read and show image:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def show_image_plt(img, title='Image', cmap=None):
    plt.figure(figsize=(8, 6))
    if len(img.shape) == 3:
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    else:
        plt.imshow(img, cmap=cmap)
    plt.title(title)
    plt.axis('off')
    plt.show()

def show_image_cv2(img, title='Image', cmap=None):
    cv2.imshow(title, img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# Load test image
filename = 'ManWoman.png'
image = cv2.imread(filename)
if image is None:
    raise FileNotFoundError(f"Test image not found: {filename}")
```

```
show_image_plt(image, 'Man & Woman')
show_image_cv2(image, 'Man & Woman')
```

Use the image `ManWoman.png` for the following exercises, but feel free to experiment with other images.

1. Human Perception of Color, Spatial Detail, and Gamma Correction

The human visual system responds nonlinearly to light intensity. Digital imaging systems use gamma correction to compensate for this nonlinearity and display characteristics. The correction follows the power law:

$$I_{out} = I_{in}^{\gamma}$$

```
def apply_gamma(image, gamma):
    # TODO: Implement gamma correction
    # 1. Convert to float32 and normalize to [0,1]
    # 2. Apply power-law transformation
    # 3. Scale back to [0,255] and convert to uint8
    return np.uint8(corrected_image)

def plot_gamma_curve(gamma):
    """Plot the gamma correction curve with reference line"""
    x = np.linspace(0, 1, 256)
    y = np.power(x, gamma)
    plt.plot(x, x, '--', label='Linear')
    plt.plot(x, y, label=f'γ = {gamma}')
    plt.xlabel('Input Intensity')
    plt.ylabel('Output Intensity')
    plt.legend()
    plt.grid(True)
    plt.show()
```

Questions:

1.1. Implement the gamma correction function apply_gama and test the results by calculating the output values for input intensities [64, 128, 192] and γ = 2.0. Present the results in a table.

1.2. The human eye is approximately logarithmic in its response to light. Demonstrate how this relates to gamma correction by comparing the histogram of an image before and after applying γ = 2.2 (the typical display gamma).

2. Colorspaces (RGB and YCbCr)

Different color spaces serve different purposes in image processing. RGB matches display hardware while YCbCr separates luminance and chrominance components. The transformation follows:

$$\begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Experiment with color space transformations using these functions:

```
ycrcb = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)
y, cr, cb = cv2.split(ycrcb)

def display_channels(channels, titles):
    plt.figure(figsize=(15, 5))
    for i, (ch, title) in enumerate(zip(channels, titles)):
        plt.subplot(1, 3, i+1)
        plt.imshow(ch, cmap='gray')
        plt.title(title)
        plt.axis('off')
    plt.tight_layout()
    plt.show()
```

Questions:

2.1. Compare the histogram spread of the Y channel versus R, G, and B channels. Why is the Y channel typically used for edge detection?

2.2. If Cb and Cr channels are subsampled by a factor of 2, calculate the memory savings. At what point does this subsampling become visually noticeable if the subsampling factor is increased?
Hint: use the subsampling function cv.resize() to test this.

3. Image Quantization and Sampling

Digital images require discrete representation in both spatial coordinates and intensity values. The number of quantization levels directly affects both image quality and storage requirements.

```
def quantize(image, bits):
    quantized_image = image.copy().astype(np.float32)
    levels = 2**bits
    div = 256 / levels
    quantized_image = (quantized_image / div).astype(np.uint8) * div
    return quantized_image
```

Questions:

3.1. Quantize the image to 4 bits and calculate the theoretical compression ratio achieved.

3.2. Plot the intensity distribution before and after 3-bit quantization. Explain the histogram's structure.

4. Point Spread Functions and Convolution

Point spread functions (PSF) describe how imaging systems blur each input point. For shift-invariant systems, this leads to convolution operations. The discrete convolution is given by:

$$g(x,y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} h(i,j) f(x-i, y-j)$$

Experiment with different kernels and analyze their effects:

```
# Common kernels for experimentation
box_kernel = np.ones((3,3)) / 9.0
# 2D Gaussian kernel implemented using matrix multiplication of 1D kernels
gauss_kernel = cv2.getGaussianKernel(5, 1.0) @ cv2.getGaussianKernel(5, 1.0).T
# 2D Gaussian kernel implemented using cv2.GaussianBlur
gauss_kernel_cv = cv2.GaussianBlur(image, (5,5), 1.0)

# Convolution example
def apply_convolution(image, kernel):
    return cv2.filter2D(image, -1, kernel)

# Separable convolution example
def apply_separable_filter(image, kernel_1d):
    """Apply separable filter using 1D kernel"""
    temp = cv2.filter2D(image, -1, kernel_1d.reshape(1, -1))
    return cv2.filter2D(temp, -1, kernel_1d.reshape(-1, 1))
```

Detecting edges can be done using the gradient magnitude.

```
gx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
magnitude_cv = np.sqrt(gx**2 + gy**2)

# Normalize magnitude
magnitude_cv = cv2.normalize(magnitude_cv, None, 0, 255, cv2.NORM_MINMAX)
```

Questions:

4.1. Compare the execution time of separable versus non-separable Gaussian filtering. When does separability provide significant advantage?
Hint: use the time.time() function to measure execution time.

4.2. Create a function that enhances an image's horizontal edges while suppressing vertical ones. Explain its structure.*hint: use the cv2.Sobel() function to detect edges.*

4.3 Show the result of applying the magnitude gradient kernel to the image.

5. Images as Stacked Vectors and Matrix Operations

Images can be reshaped into vectors by stacking columns or rows, enabling efficient matrix operations. Consider an N×N image transformed into an N²×1 vector:

$$\mathbf{f} = [f(0,0), f(1,0), \ldots, f(N-1,0), f(0,1), \ldots, f(N-1,N-1)]^T$$

Try this vectorized approach:

```
# Convert image to vector form
img_vector = image.reshape(-1, 1)

# Perform operation using matrix multiplication
# transformed = ...

# Reshape back to image shape
result = transformed.reshape(image.shape)
```

Questions:
5.1. Implement image brightness and contrast adjustment using vector operations. Compare the computational efficiency with element-wise operations where nested loops are used, by measuring the execution time.

5.2. Given a 512×512 image, calculate the memory requirements for storing the explicit transformation matrix H for a 3×3 convolution operation.

6. Pixel-based Processing

Each pixel can be processed independently through point operations. For a given input intensity r, the output s is:

$$s = T(r)$$

Where T(r) could be linear, logarithmic, power-law, or other nonlinear functions. The function T(r) could either be implemented as a lookup table or as a direct computation.

```
def create_lookup_table(function_type, params):
    lut = np.zeros(256, dtype=np.uint8)
    # TODO: Implement a intensity transformations
    # Use numpy functions for vectorized operation
    return lut

def direct_computation_T(r):
    # TODO: Implement a intensity transformations
    # Use numpy functions for vectorized operation
    return r
```

```
# Lookup table example
lut = create_lookup_table('log', {'c': 1.0})
output = lut[image]

# Direct computation example
output = direct_computation_T(image)
```

Questions:

6.1. For your implementation, which is more efficient: direct computation or lookup tables? Quantify the difference. *Hint: Use numpy functions for vectorized operation and the time.time() function to measure execution time.*

6.2. Design a logarithmic transformation that enhances contrast in the middle intensity range while compressing shadows and highlights. Plot the transfer function.

7. Elementary Images and Linear Superposition

Any image can be expressed as a weighted sum of elementary images formed by outer products of vectors:

$$f = \sum_{i=1}^{N} \sum_{j=1}^{N} g_{ij} \mathbf{u}_i \mathbf{v}_j^T$$

Study this decomposition:

```
def create_basis_vectors(size, type='ramp'):
    if type == 'ramp':
        return np.linspace(0, 1, size)
    elif type == 'step':
        return np.heaviside(np.arange(size) - size/2, 1)
    elif type == 'sine':
        return np.sin(2 * np.pi * np.arange(size) / size)

# Example: Create and display elementary image
size = 64
u = create_basis_vectors(size, 'ramp')
v = create_basis_vectors(size, 'step')
elementary_image = np.outer(u, v)
show_image(elementary_image, 'Elementary Image', 'gray')
```

Questions:

7.1. Create four different elementary images of size 16x16using combinations of ramp, step, and sine basis vectors. How do they capture different image features?

7.2. Select a simple geometric shape in an image of size 16x16 and approximate it using only three elementary images by finding the best respective weights.

8. Transforms and Frequency Domain

The frequency domain reveals periodic patterns and enables efficient filtering. The 2D Fourier transform gives:

$$F(u,v) = \frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi(\frac{ux}{M}+\frac{vy}{N})}$$

Analyze images in the frequency domain:

```
def display_spectrum(image_gray):
    """Display the magnitude spectrum in logarithmic scale"""
    f = np.fft.fft2(image_gray)
    fshift = np.fft.fftshift(f)
    magnitude = np.log1p(np.abs(fshift))
    show_image(magnitude, 'Magnitude Spectrum', 'gray')

def ideal_lowpass(shape, cutoff):
    """Create ideal lowpass filter in frequency domain"""
    rows, cols = shape
    crow, ccol = rows//2, cols//2
    mask = np.zeros((rows, cols))
    center = [crow, ccol]
    for i in range(rows):
        for j in range(cols):
            if np.sqrt((i-center[0])**2 + (j-center[1])**2) < cutoff:
                mask[i,j] = 1
    return mask
```

Questions:

8.1. Apply the FFT to your image and identify the frequencies corresponding to dominant edges. Where are they located?

8.2. Remove the high frequencies from the FFT, and inverse transform back the transformed images to the spatial domain. Compare the result with the original image. What happens to the image content?

8.3. Remove the low frequencies from the FFT and inverse transform back to the spatial domain. Compare the result with the original image.
hint: use the results of the previous question and subtract the results from the original image.

9. Histograms and Image Enhancement

Histograms represent the frequency distribution of pixel intensities, providing insights into image characteristics and guiding enhancement operations. The cumulative histogram serves as the basis for histogram equalization:

$$s_k = T(r_k) = \frac{L-1}{n} \sum_{j=0}^{k} n_j$$

where L is the number of intensity levels and n is the total number of pixels.

```
def compute_histogram(image):
    """Get normalized histogram"""
    hist = cv2.calcHist([image], [0], None, [256], [0,255])
    return hist / hist.sum()

def cumulative_histogram(hist):
    """Convert histogram to cumulative form"""
    return np.cumsum(hist)
```

Questions:

9.1. Apply and plot the histogram and cumulative histogram functions to your image. What can be said about the image content based on the histogram and cumulative histogram?

9.2. Apply histogram equalization the image after you have reduced its contrast. Show the result and compare it to the original image. *hint: use the point operations and the cumulative histogram as the transformation function. Validate your results with those of cv2.equalizeHist().*

10. Noise and Noise Reduction

Image noise takes various forms including Gaussian, salt-and-pepper, and speckle noise. Different filtering approaches are suited to different noise types.

```
# Add different types of noise for testing
noisy_gaussian = image + np.random.normal(0, 25, image.shape).astype(np.uint8)
noisy_sp = image.copy()
mask = np.random.random(image.shape[:2]) < 0.02
noisy_sp[mask] = [0, 0, 0]
mask = np.random.random(image.shape[:2]) < 0.02
noisy_sp[mask] = [255, 255, 255]
```

Study how different filters perform:

```
def median_filter(image, ksize=3):
    return cv2.medianBlur(image, ksize)

def bilateral_filter(image):
    return cv2.bilateralFilter(image, 9, 75, 75)
```

Questions:
10.1 Add noise to the filter and measure the PSNR of the results. Change the noise levels such that the same PSNR is achieved for both filters, and present the parameters used.
10.2. Compare the effectiveness of mean filtering versus median filtering for salt-and-pepper noise, and gaussian noise. Quantify the difference using PSNR.
hint: use the cv2.PSNR() function.

11. Morphological Filtering

Morphological operations process images based on shapes. The fundamental operations are erosion and dilation, defined by:
Erosion:

$$E(A, B) = \{z | B_z \subseteq A\}$$

Dilation:

$$D(A, B) = \{z | B_z \cap A \neq \emptyset\}$$

```
# Create structuring elements of different shapes
cross_kernel = np.array([[0,1,0], [1,1,1], [0,1,0]], np.uint8)
disk_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))

def morph_gradient(image):
    """
    Compute morphological gradient.
    Returns: Dilation - Erosion
    """
    dilated = cv2.dilate(image, cross_kernel)
    eroded = cv2.erode(image, cross_kernel)
    return dilated - eroded
```

Questions:
11.1. Develop a sequence of morphological operations to detect and count objects in a binary image. Test with different object shapes.

11.2. Compare the morphological gradient with the Sobel based gradient in Section 4 for edge detection. When is each method preferable?

Final Notes

- Present your results in a short report, including answers with clear support of conclusions, code and anything else you consider relevant.
- Relate your findings to the theoretical concepts from lectures when applicable.
- Submit your report to the handin box in Moodle.